

TARGET URL: http://testphp.vulnweb.com/	SCAN DATE: 2025-11-28 12:17:15	REPORT GENERATED: 2025-11-28 12:17:15
--	-----------------------------------	--

Scan Summary

Risk Level	Count
High	0
Medium	3
Low	3
Informational	0
Total	6

Detailed Findings

Absence of Anti-CSRF Tokens

Medium Risk

Predicted Score: 12.77

Confidence: Low

URL: http://testphp.vulnweb.com/

Description:

No Anti-CSRF tokens were found in a HTML submission form.

A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf.

CSRF attacks are effective in a number of situations, including:

- * The victim has an active session on the target site.
- * The victim is authenticated via HTTP auth on the target site.
- * The victim is on the same local network as the target site.

CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.

Solution:

Phase: Architecture and Design

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.

For example, use anti-CSRF packages such as the OWASP CSRFGuard.

Phase: Implementation

Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script.

Phase: Architecture and Design

Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330).

Note that this can be bypassed using XSS.

Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation.

Note that this can be bypassed using XSS.

Use the ESAPI Session Management control.

This control includes a component for CSRF.

Do not use the GET method for any request that triggers a state change.

Phase: Implementation

Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.

Reference:

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

<https://cwe.mitre.org/data/definitions/352.html>

Missing Anti-clickjacking Header

Medium Risk

Predicted Score: 11.0

Confidence: Medium

URL: http://testphp.vulnweb.com/

Description:

The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

Solution:

Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.

If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.

Reference:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options>

Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)

Low Risk

Predicted Score: 9.88

Confidence: Medium

URL: http://testphp.vulnweb.com/

Description:

The web/application server is leaking information via one or more "X-Powered-By" HTTP response headers. Access to such information may facilitate attackers identifying other frameworks/components your web application is reliant upon and the vulnerabilities such components may be subject to.

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to suppress "X-Powered-By" headers.

Reference:

https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework

<https://www.troyhunt.com/shhh-dont-let-your-response-headers/>

Server Leaks Version Information via "Server" HTTP Response Header Field

Low Risk

Predicted Score: 9.88

Confidence: High

URL: http://testphp.vulnweb.com/

Description:

The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.

Reference:

<https://httpd.apache.org/docs/current/mod/core.html#servertokens>

[https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552\(v=pandp.10\)](https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10))

<https://www.troyhunt.com/shhh-dont-let-your-response-headers/>

User Controllable HTML Element Attribute (Potential XSS)

Predicted Score: 9.39

Confidence: Low

URL: http://testphp.vulnweb.com/guestbook.php

Description:

This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.

Solution:

Validate all input and sanitize output it before writing to any HTML attributes.

Reference:

https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

Content Security Policy (CSP) Header Not Set

Medium Risk

Predicted Score: 5.25

Confidence: High

URL: <http://testphp.vulnweb.com/>

Description:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Reference:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>

https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

<https://www.w3.org/TR/CSP/>

<https://w3c.github.io/webappsec-csp/>

<https://web.dev/articles/csp>

<https://caniuse.com/#feat=contentsecuritypolicy>

<https://content-security-policy.com/>

X-Content-Type-Options Header Missing

Low Risk

Predicted Score: 5.25

Confidence: Medium

URL: http://testphp.vulnweb.com/

Description:

The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.

Solution:

Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.

If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

Reference:

[https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941\(v=vs.85\)](https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85))

<https://owasp.org/www-community/Security-Headers>

Authentication Request Identified

Predicted Score: Unmapped

Confidence: Low

URL: http://testphp.vulnweb.com/secured/newuser.php

Description:

The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.

Solution:

This is an informational alert rather than a vulnerability and so there is nothing to fix.

Reference:

<https://www.zaproxy.org/docs/desktop/addons/authentication-helper/auth-req-id/>

Charset Mismatch (Header Versus Meta Content-Type Charset)

Predicted Score: Unmapped

Confidence: Low

URL: http://testphp.vulnweb.com/

Description:

This check identifies responses where the HTTP Content-Type header declares a charset different from the charset defined by the body of the HTML or XML. When there's a charset mismatch between the HTTP header and content body Web browsers can be forced into an undesirable content-sniffing mode to determine the content's correct character set.

An attacker could manipulate content on the page to be interpreted in an encoding of their choice. For example, if an attacker can control content at the beginning of the page, they could inject script using UTF-7 encoded text and manipulate some browsers into interpreting that text.

Solution:

Force UTF-8 for all text content in both the HTTP header and meta tags in HTML or encoding declarations in XML.

Reference:

https://code.google.com/p/browsersec/wiki/Part2#Character_set_handling_and_detection

Modern Web Application

Predicted Score: Unmapped

Confidence: Medium

URL: http://testphp.vulnweb.com/AJAX/index.php

Description:

The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.

Solution:

This is an informational alert and so no changes are required.